
```
def displayDisk(width):
    """Display a disk of the given width. A width of 0 means no disk."""
    emptySpace = ' ' * (TOTAL_DISKS - width)
    if width == 0:
        # Display a pole segment without a disk:
        print(f'{emptySpace}||{emptySpace}', end='')
    else:
        # Display the disk:
        disk = '@' * width
        numLabel = str(width).rjust(2, '_')
        print(f"{emptySpace}{disk}{numLabel}{disk}{emptySpace}", end='')
```

We represent a disk using a leading empty space, a number of @ characters equal to the disk width, two characters for the width (including an underscore if the width is a single digit), another series of @ characters, and then the trailing empty space. To display just the empty pole, all we need are the leading empty space, two pipe characters, and trailing empty space. As a result, we'll need six calls to `displayDisk()` with six different arguments for `width` to display the following tower:

```
  ||
  @_1@
 @@_2@@
 @@@_3@@@
 @@@@_4@@@@
 @@@@@_5@@@@@
```

Notice how the `displayTowers()` and `displayDisk()` functions split the responsibility of displaying the towers. Although `displayTowers()` decides how to interpret the data structures that represent each tower, it relies on `displayDisk()` to actually display each disk of the tower. Breaking your program into smaller functions like this makes each part easier to test. If the program displays the disks incorrectly, the problem is likely in `displayDisk()`. If the disks appear in the wrong order, the problem is likely in `displayTowers()`. Either way, the section of code you'll have to debug will be much smaller.

To call the `main()` function, we use a common Python idiom:

```
# If this program was run (instead of imported), run the game:
if __name__ == '__main__':
```